

ARQUITECTURA DEL COMPUTADOR

INFORME PRÁCTICA DE LABORATORIO 2

Nombre: Victor Pinto, C.I: 31709216

Pregunta 1

¿Qué diferencias existen entre registros temporales (\$t0-\$t9) y registros guardados (\$s0-\$s7) y cómo se aplicó esta distinción en la práctica?

La principal diferencia entre registros temporales y guardados radica en el principio de que los registros temporales no se preservan entre llamadas a funciones, lo que conlleva que el contenido de dichos registros pueda ser alterado, mientras que los registros guardados deben preservarse durante y liberarse durante un mismo procedimiento, de tal forma que quien haya llamado a la función pueda usar los registros \$s0-\$s7 sin preocuparse por la volatilidad de su contenido. Los registros temporales, sin embargo, al no ser preservados, se verán alterados durante la función y el llamador habrá perdido su contenido.

En la implementación de Quicksort y Mergesort, las funciones principales llaman constantemente a otras funciones, como en la partición o el *swap* del Quicksort. Es necesario preservar la pila para los registros y que puedan ser usados en dichas funciones, y a su vez guardar el contenido después de terminadas.

Pregunta 2

¿Qué diferencias existen entre los registros \$a0-\$a3, \$v0-\$v1, \$ra y cómo se aplicó esta distinción en la práctica?

Los registros \$a0-\$a3 están dedicados a los parámetros de un procedimiento. Estos almacenan los parámetros que una función requiere, guardándolos justo antes de llamarla. Por otro lado, los registros \$v0-\$v1 se utilizan para almacenar los valores que, al final de una función, son retornados. Con el valor guardado en el registro de retorno, el registro \$ra contiene la última dirección antes de haber sido llamada la función, con lo cual se regresa a la invocación, junto con el valor retornado.

En la implementación, Quicksort y Mergesort requieren parámetros para los arreglos, usando \$a0-\$a3; también se necesita, en caso de Quicksort, el retorno del pivote, para lo cual se emplea los registros \$v0-\$v1; y para regresar a la dirección antes de entrar a las funciones que ambos algoritmos utilizan, \$ra se encarga de situar y retornar a tal dirección.

Pregunta 3

¿Cómo afecta el uso de registros frente a memoria en el rendimiento de los algoritmos de ordenamiento implementados?

En MIPS32, se puede acceder a los registros casi que de manera inmediata. Sus operaciones no son lentas y están a disposición rápidamente para el procesador. En cambio, el acceso a memoria, presente en ambas implementaciones para el manejo de los arreglos y subarreglos, son operaciones que tardan más ciclos de reloj, pues requieren la búsqueda de los datos en memoria.

Pregunta 4

¿Qué impacto tiene el uso de estructuras de control (bucles anidados, saltos) en la eficiencia de los algoritmos en MIPS32?

Naturalmente, los bucles anidados provocan una menor eficiencia en algoritmos de ordenamiento. En el caso de Quicksort, este emplea la recursión para ordenar un arreglo, mientras que la implementación iterativa de Mergesort sí tiene un bucle anidado; no obstante, no impacta tanto pues la estructura de este algoritmo garantiza tiempos eficientes. Los saltos entre funciones crean constantemente nuevas instancias, y requieren guardar las direcciones desde donde fueron invocadas. Nótese que, en el camino de datos segmentado, las instrucciones posteriores a un salto deben detenerse, creando burbujas. MIPS resuelve esto haciendo uso de predicciones.

Pregunta 5

¿Cuáles son las diferencias de complejidad computacional entre el algoritmo Quicksort y el algoritmo Mergesort? ¿Qué implicaciones tiene esto para la implementación en un entorno MIPS32?

Mergesort garantiza una complejidad $O(n \log n)$ por la manera en que divide el arreglo original en subarreglos. Quicksort, por otra parte, en el caso promedio también comparte la complejidad algorítmica de Mergesort, pero debido a que es dependiente de la elección del pivote, puede llegar a ser de $O(n^2)$. En el entorno de MIPS, esto implica que Quicksort puede llegar a acumular muchas instancias producto de la recursión.

Pregunta 6

¿Cuáles son las fases del ciclo de ejecución de instrucciones en la arquitectura MIPS32 (camino de datos)? ¿En qué consisten?

Una instrucción atraviesa un camino que consta de 5 fases:

1. El **PC** (*Program Counter*) envía la dirección de la instrucción a la Memoria de instrucciones y se aumenta en 4.
2. La **Memoria de instrucciones** decodifica la instrucción según sus campos y los envía al Banco de registros.
3. Dependiendo del tipo de instrucción, del banco se leen los registros con los que se opera.
4. Se mandan los valores a la **ALU**, el componente que se encarga de realizar operaciones aritméticas.

5. Finalmente, se llega a la **Memoria de datos**, donde se guardan o cargan datos de la memoria.

Pregunta 7

¿Qué tipo de instrucciones se usaron predominantemente en la práctica (R, I, J) y por qué?

Principalmente se usan las instrucciones de tipo J, para realizar los saltos entre funciones, y las de tipo R que permiten operar y comparar registros; algo sumamente importante para controlar ciertos lazos y saltos condicionales.

Pregunta 8

¿Cómo se ve afectado el rendimiento si se abusa del uso de instrucciones de salto (j, beq, bne) en lugar de usar estructuras lineales?

Las instrucciones de tipo J paralizan el *pipeline*. Aun cuando MIPS usa la predicción para apaciguar el impacto, el constante salto entre funciones requiere esperar a que estas finalicen y continuar con el programa. Las estructuras lineales permitirían que el programa se ejecute con mayor velocidad al no depender de la espera por los saltos.

Pregunta 9

¿Qué ventajas ofrece el modelo RISC de MIPS en la implementación de algoritmos básicos como los de ordenamiento?

El modelo RISC establece el diseño de instrucciones cortas y sencillas que se ejecutan en una sola acción. La suma de todas ellas hace un programa. La ventaja que ofrecen estas en la implementación de algoritmos de ordenamiento es la precisión que tiene cada instrucción, lo que permite hallar e identificar errores en lugares puntuales, aunque conlleven más líneas de código. Un catálogo corto de instrucciones también facilita la segmentación, sumamente importante para garantizar eficiencia.

Pregunta 10

¿Cómo se usó el modo de ejecución paso a paso (*Step*, *Step Into*) en MARS para verificar la correcta ejecución del algoritmo?

Este modo permite ver, a cada instante, la ejecución de las instrucciones que componen a los algoritmos. Se usó para observar cómo se saltaba entre funciones, y el comportamiento que estas mostraban. También con los arreglos y subarreglos, que son tratados en dichas funciones.

Pregunta 11

¿Qué herramienta de MARS fue más útil para observar el contenido de los registros y detectar errores lógicos?

La herramienta *Step, Step Into* muestra el contenido de los registros junto a las instrucciones, en tiempo real. Es la manera más cómoda de vigilar la evolución y el cambio de los registros entre saltos e instrucciones.

Pregunta 12

¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo R? (por ejemplo: `add`)

MIPS trae integrada una herramienta llamada *MIPS X-Ray*, que permite observar el camino de datos de cualquier instrucción. Basta con escribir un programa que contenga la instrucción deseada, ir a la pestaña de herramientas y seleccionar el *MIPS X-Ray*. Al conectarse y compilar, la herramienta describe los caminos que toman los registros y el paso por los componentes del camino de datos. `add` utiliza dos registros, que son leídos del banco y cuyo contenido es operado en la ALU. Este proceso es denotado por diferentes colores en el *MIPS X-Ray*.

Pregunta 13

¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo I? (por ejemplo: `lw`)

Al igual que con la instrucción `add`, la herramienta *MIPS X-Ray* muestra el camino que se lleva a cabo al momento de ejecutar una instrucción tipo I. En el caso de `lw`, la herramienta muestra cómo se decodifica la instrucción, se extiende el signo y se calcula la dirección de memoria a acceder.

Pregunta 14

Justificar la elección del algoritmo alternativo.

Resultó más cómodo escoger a Quicksort como algoritmo recursivo por su familiaridad. Es un algoritmo con el que ya se había trabajado antes, en MIPS32. Se decidió hacer una versión iterativa de Mergesort pues se considera que su recursión no resulta tan crítica como en su algoritmo hermano.

Pregunta 15

Análisis y Discusión de los Resultados.

Se probaron ambos algoritmos de ordenamiento con arreglos de tamaño variable. El resultado fue óptimo y esperado. También se probaron los casos con arreglos unitarios y con elementos repetidos, donde la salida fue la esperada. Debido a que Merge Sort es iterativo, y al contener un par de ciclos, su rendimiento es menor que Quicksort.